

Physics Informed Extreme Learning Machine (PIELM): A Rapid Method for the Numerical Solution of Partial Differential Equations

Summary of Dwivedi and Srinivasan, Neurocomputing 2020

1 Introduction

Physics Informed Extreme Learning Machine (PIELM) represents a significant advancement in solving partial differential equations (PDEs) through neural network approaches. Developed as a hybrid of Extreme Learning Machine (ELM) and Physics Informed Neural Networks (PINNs), PIELM addresses several key limitations of traditional numerical methods and deep learning approaches for solving PDEs.

Traditional numerical methods for solving PDEs (such as Finite Element Method, Finite Difference Method, and Finite Volume Method) face two primary challenges: (1) difficulty in handling complex computational domains where grid generation becomes infeasible, and (2) discretization errors that create discrepancies between the mathematical nature of actual PDEs and their approximate difference equations.

Neural network approaches, particularly Physics Informed Neural Networks (PINNs), offer promising alternatives that can handle complex geometries in a meshfree fashion. However, PINNs suffer from slow learning processes, lack of guarantees for optimization convergence, and the absence of systematic ways to determine appropriate architecture or data requirements for a given problem.

PIELM addresses these limitations by combining the physics-informed loss function approach of PINNs with the computational efficiency of Extreme Learning Machines, resulting in a method that is both accurate and extremely fast for solving linear PDEs.

2 Mathematical Foundations

2.1 Extreme Learning Machine (ELM)

ELM is a single-layer feedforward neural network with the following distinctive characteristics:

- Random assignment of input layer weights and biases that remain fixed (non-trainable)
- Analytical determination of output weights
- Much faster training compared to gradient-based approaches

For a standard ELM with N^* neurons in the hidden layer, given an input vector $\vec{x}$ of size $m \times 1$, the output is calculated as:

$$\vec{y} = H(\vec{x})\vec{c} \quad (1)$$

where $H = [\vec{h}(\vec{x}_1), \vec{h}(\vec{x}_2), \dots, \vec{h}(\vec{x}_N)]^T$, with $\vec{h}(\vec{x}_k) = [\phi(\vec{w}_1^T \vec{x}_k + b_1), \phi(\vec{w}_2^T \vec{x}_k + b_2), \dots, \phi(\vec{w}_{N^*}^T \vec{x}_k + b_{N^*})]$, and $\vec{c} = [c_1, c_2, \dots, c_{N^*}]^T$ is the vector of output layer weights.

For a training dataset $\{(\vec{x}_k, y_k)\}_{k=1}^N$, the optimal output weights are determined analytically using:

$$\vec{c} = \text{pinv}(H)\vec{T} \quad (2)$$

where $\text{pinv}(H)$ is the pseudo-inverse of H and $\vec{T}$ represents the target values.

2.2 Physics Informed Neural Network (PINN)

PINNs embed physics knowledge into the neural network training process by including the PDE residuals in the loss function. For a PDE of the form:

$$\frac{\partial}{\partial t}u(x, t) + \mathcal{N}u(x, t) = R(x, t), \quad (x, t) \in \Omega \quad (3)$$

$$u(x, t) = B(x, t), \quad (x, t) \in \partial\Omega \quad (4)$$

$$u(x, 0) = F(x), \quad x \in (x_L, x_R) \quad (5)$$

where $\mathcal{N}$ is a differential operator, the PINN approximates $u(x, t)$ with the network output $f(x, t)$. The errors in approximating the PDE, boundary conditions (BCs), and initial conditions (ICs) are defined as:

$$\vec{\xi}_f = \frac{\partial \vec{f}}{\partial t} + \mathcal{N}\vec{f} - \vec{R} \quad (6)$$

$$\vec{\xi}_{bc} = \vec{f} - \vec{B} \quad (7)$$

$$\vec{\xi}_{ic} = \vec{f}(\cdot, 0) - \vec{F} \quad (8)$$

The loss function minimized during training is:

$$J = \frac{\vec{\xi}_f^T \vec{\xi}_f}{2N_f} + \frac{\vec{\xi}_{bc}^T \vec{\xi}_{bc}}{2N_{bc}} + \frac{\vec{\xi}_{ic}^T \vec{\xi}_{ic}}{2N_{ic}} \quad (9)$$

where N_f , N_{bc} , and N_{ic} refer to the number of collocation points, boundary condition points, and initial condition points, respectively.

3 Physics Informed Extreme Learning Machine (PIELM)

3.1 Algorithm Description

PIELM combines the physics-informed loss function approach with the computational efficiency of ELM. The key innovation is applying the extreme learning framework to solve PDEs by:

1. Assigning input layer weights randomly
2. Incorporating the physics of the PDE in the loss function
3. Analytically calculating the output layer weights

For a 1D unsteady PDE problem, if we define $\vec{\chi} = [x, t, 1]^T$, $\vec{m} = [m_1, m_2, \dots, m_{N^*}]^T$, $\vec{n} = [n_1, n_2, \dots, n_{N^*}]^T$, $\vec{b} = [b_1, b_2, \dots, b_{N^*}]^T$ and $\vec{c} = [c_1, c_2, \dots, c_{N^*}]^T$, then the output of the k^{th} hidden neuron is $h_k = \phi(z_k)$, where $z_k = [m_k, n_k, b_k]\vec{\chi}$ and $\phi = \tanh$ is the nonlinear activation function.

The PIELM output is given by:

$$f(\vec{\chi}) = \vec{h}\vec{c} \quad (10)$$

The partial derivatives needed for the PDE are calculated exactly (without discretization error) as:

$$\frac{\partial^p f_k}{\partial x^p} = m_k^p \frac{\partial^p \phi}{\partial z^p} \quad (11)$$

$$\frac{\partial f_k}{\partial t} = n_k \frac{\partial \phi}{\partial z} \quad (12)$$

If an ELM with N^* hidden nodes can solve the PDE with zero error, it implies that there exist $\vec{c}$, $\vec{m}$, $\vec{n}$ and $\vec{b}$ such that:

$$\vec{\xi}_f = \vec{0} \quad (13)$$

$$\vec{\xi}_{bc} = \vec{0} \quad (14)$$

$$\vec{\xi}_{ic} = \vec{0} \quad (15)$$

These equations can be collectively represented as:

$$H\vec{c} = \vec{K} \quad (16)$$

where the form of H and $\vec{K}$ depends on the differential operator $\mathcal{N}$, the boundary conditions B , and the initial conditions F . The output layer weights are obtained using $\vec{c} = \text{pinv}(H)\vec{K}$.

Algorithm 1 PIELM Algorithm

- 1: Assign the input layer weights randomly
 - 2: Depending on the PDE and the initial and boundary conditions, find expressions for $\vec{\xi}_f$, $\vec{\xi}_{bc}$ and $\vec{\xi}_{ic}$
 - 3: Assemble the three sets of equations in the form of $H\vec{c} = \vec{K}$
 - 4: Calculate output layer weight vector using $\vec{c} = \text{pinv}(H)\vec{K}$
-

3.2 Architecture Estimation

One significant advantage of PIELM is its ability to provide a criterion for determining the optimal number of neurons in the hidden layer. Setting the number of hidden units equal to the number of constraints ($N^* = N_f + N_{bc} + N_{ic}$) typically yields optimal results. The accuracy of prediction is sensitive to the number of neurons for $N < N^*$, but for $N > N^*$, the accuracy doesn't improve significantly with an increase in the number of neurons.

3.3 Limitations

The proposed PIELM applies only to linear PDEs or linear differential operators with constant or spatially varying coefficients. For nonlinear operators, the resulting system of equations becomes nonlinear and cannot be solved using simple matrix inversion methods. For example, in the nonlinear inviscid Burgers' equation:

$$u_t + uu_x = 0 \quad (17)$$

The residual equations transform into nonlinear algebraic equations which cannot be solved using pseudo-inverse operations.

4 Distributed PIELM (DPIELM)

To address PIELM’s limitation in handling PDEs with sharp gradient solutions, the authors developed a distributed version called DPIELM. Inspired by traditional Finite Element Methods, DPIELM divides the computational domain into multiple sub-domains, with each sub-domain represented by a separate PIELM. These individual PIELM representations are then connected through interface constraints that ensure continuity and/or differentiability across domain boundaries.

The implementation of DPIELM involves:

1. Dividing the computational domain into non-overlapping cells
2. Installing a separate PIELM in each cell
3. Formulating appropriate interface regularization conditions
4. Assembling a global system of equations
5. Solving for all output layer weights simultaneously

For a computational domain Ω divided into N_c non-overlapping cells $\{\Omega_i\}_{i=1}^{N_c}$, with each cell having its own PIELM $M^{(i)}$ and output $f^{(i)}$, the interface regularization depends on the PDE being solved:

- For first-order PDEs (e.g., advection equations), continuity constraints (C^0 continuity) at cell interfaces are sufficient
- For second-order PDEs (e.g., diffusion equations), both continuity and differentiability constraints (C^1 continuity) are required

These interface constraints effectively stitch together the piecewise neural network representations, allowing the method to handle sharp gradient solutions while maintaining physical consistency across the entire domain.

5 Numerical Examples and Results

5.1 Performance on Linear PDEs

The authors tested PIELM on various linear and quasi-linear PDEs, including:

- 1D steady advection, diffusion, and advection-diffusion equations
- 2D steady advection and diffusion equations in complex geometries
- 1D unsteady advection equations with constant and variable coefficients

For 1D steady cases, PIELM achieved accuracy on the order of 10^{-3} to 10^{-6} with computational times on the order of milliseconds. For 2D steady cases on complex geometries (including a star-shaped domain and the map of Illinois), PIELM achieved accuracies of 10^{-4} to 10^{-7} with computational times of a few seconds.

Comparatively, unified deep ANNs (similar to PINNs) required approximately 8 hours of computation time and 5,500 data points to achieve an accuracy of 10^{-5} for 2D diffusion problems. PIELM solved the same problems with only 1,161 data points and achieved accuracy levels of 10^{-4} to 10^{-6} in just a few seconds.

For 1D unsteady advection problems, PIELM correctly predicted the exact solutions for both constant and variable coefficient cases with a learning time of 2-3 seconds. Unlike conventional numerical methods, PIELM does not suffer from CFL (Courant-Friedrichs-Lewy) stability constraints that limit time step sizes in traditional approaches.

5.2 Performance on PDEs with Sharp Gradient Solutions

Regular PIELM (and PINNs) struggle with representing functions having sharp gradients or discontinuities. The authors demonstrated this limitation on several test cases:

- Representation of 1D sharp and discontinuous functions
- Representation of a sharp-peaked 2D Gaussian
- 1D steady advection-diffusion with low diffusion coefficient
- 1D unsteady advection of a sharp-peaked Gaussian
- 1D and 2D unsteady advection-diffusion equations with low diffusion

In contrast, DPIELM performed successfully on all these challenging cases. For the 2D unsteady advection-diffusion equation with a low diffusion coefficient ($\nu = 0.005$), DPIELM achieved accuracy comparable to sophisticated conventional numerical techniques (Discontinuous Galerkin methods), marking the first successful application of an ELM-based algorithm to such problems.

DPIELM also successfully solved the advection of high-frequency wave packets that deep PINNs failed to capture, demonstrating the advantage of efficient neural network design over simply increasing the depth and size of networks.

6 Implications and Advantages

The PIELM and DPIELM approaches offer several significant advantages:

6.1 Computational Efficiency

PIELM is extremely fast compared to both traditional numerical methods and deep PINNs. The analytical determination of output weights eliminates the need for iterative optimization, resulting in training times that are orders of magnitude faster than gradient-based approaches.

6.2 Meshfree Approach for Complex Geometries

As meshfree methods, PIELM and DPIELM can handle complex geometries where traditional grid generation becomes challenging or infeasible. This was demonstrated through successful applications on star-shaped domains and the irregular outline of the state of Illinois.

6.3 Reduced Numerical Artifacts

PIELM significantly reduces numerical artifacts like false diffusion that commonly occur in traditional upwind discretization schemes, owing to its physics-informed basis functions.

6.4 Systematic Architecture Selection

Unlike deep neural network approaches that rely on trial and error for architecture selection, PIELM provides a clear criterion for determining the optimal number of hidden neurons based on the number of constraints in the problem.

6.5 Enhanced Representation Capacity through Domain Decomposition

DPIELM extends the capabilities of PIELM to handle sharp gradient solutions through domain decomposition, without requiring deeper networks or more complex training procedures. This approach combines the efficiency of ELM with the flexibility of piecewise representations, offering a powerful alternative to both conventional numerical methods and deep learning approaches for PDEs.

7 Future Directions

The current formulation of PIELM and DPIELM is limited to linear PDEs with constant or space-varying coefficients. Two primary areas for future development include:

1. Comprehensive comparison with conventional numerical techniques in terms of speed, accuracy, and applicability across a wider range of problems
2. Extension to nonlinear PDEs, either through non-convex optimization approaches or through linearization strategies similar to explicit time-stepping methods in conventional numerical techniques

The significant computational advantages demonstrated by PIELM and DPIELM suggest that these methods have the potential to become competitive alternatives to conventional discretization techniques for solving PDEs in complex domains, particularly for time-sensitive applications where rapid solutions are essential.